Architecting Adaptive Control for Hydrogen Microgrids

A Systems Approach to Uncertainty-Aware Energy Management

In complex microgrids integrating renewables, hydrogen production, and grid constraints, classical optimization fails under uncertainty, delays, and nonlinearity. This project demonstrates a **layered R&D approach** to designing robust decision infrastructures:

- **Physics-informed device models** grounded in electrochemical principles,
- **Hierarchical control architecture** separating planning (MDP) from execution (state machines),
- **Adaptive strategies** that blend forecasting, real-time feedback, and safety constraints.

The result is a **transparent, maintainable, and explainable** system — where every decision can be traced to physical or economic logic, not black-box AI.

MDP Control Architecture Blueprint

- **Plant Layer (Physical System)**
  - Electrolysers (AEM/ALK), battery, fuel cell, PV, grid interface
  - State variables: SOC, LOH, power flows, tariffs
- **Controller Layer (Decision Logic)**
  - *Long-horizon planner*: MDP-based strategy optimizer
  - *Short-horizon executor*: Deterministic state machine
  - Dual-mode operation: target-driven vs. green-only
- **Information Flow**
  - Forecasting engine (wind, load, uncertainty)
  - Economic model (€/kWh tariffs, H production cost)
  - Safety constraints (SOC/LOH bounds, grid limits)
- **Execution Protocol**
  - Controller steps at coarse granularity (e.g., hourly)
  - Plant simulates at fine granularity (e.g., per-minute)
  - Action $\rightarrow$ State $\rightarrow$ Observation cycle enforces causality

Key Insights

- **Hybrid control beats pure ML**: Deterministic fallbacks increase trust in adaptive systems
- **Abstraction enables reuse**: Device-agnostic interfaces allow topology flexibility
- **Economics must be explicit**: Cost-per-kg H is a better objective than "efficiency"
- **Constraints are features**: Grid export limits and boiler dumps define feasible actions
- **Transparency = adoptability**: Engineers accept systems they can debug and explain

Algorithm 1 ControlSystemStep(Microgrid, Controller)

```
microgridState ← Microgrid.state
Controller.step(microgridState)
controlAction ← Controller.action
plantStepsPerControllerStep ← Controller.samplingTime / Microgrid.samplingTime
for plantStepsPerControllerStep do
    Microgrid.step(controlAction)
end for
state ← structure(controlAction, microgridState)
```

Algorithm 2 MicrogridStep(controlAction)

```
Irradiance.step()
availableSolarPower ← IRRADIANCE-TO-PV · Irradiance.irradiance
Load.step()
loadPower ← Load.power
controlActionForElectrolysers ← controlAction.electrolysersSetProductionRates
ElectrolysersGroup.step(controlActionForElectrolysers)
electrolysersTotalPower ← ElectrolysersGroup.totalPower
fuelCellOn ← controlAction.fuelCellOn
fuelCellSetPower ← loadPower
FuelCell.step(structure(fuelCellOn, fuelCellSetPower))
fuelCellPower ← FuelCell.power
remainingPower ← availableSolarPower − loadPower − electrolysersTotalPower + fuelCellPower
BatterySystem.step(remainingPower)
remainingPower ← remainingPower − BatterySystem.power
powerCut ← (remainingPower < −ϵ)
pvPower = availableSolarPower − remainingPower
HydrogenTank.step(ElectrolysersGroup.totalProductionRate − FuelCell.consumptionRate)
state ← structure(Irradiance.state, Load.state, BatterySystem.state, FuelCell.state, ElectrolysersGroup.state,
HydrogenTank.state, pvPower, remainingPower, powerCut, time)
```

Algorithm 3 ControllerStep(plantState, ElectrolysersGroupController, FuelCellController)

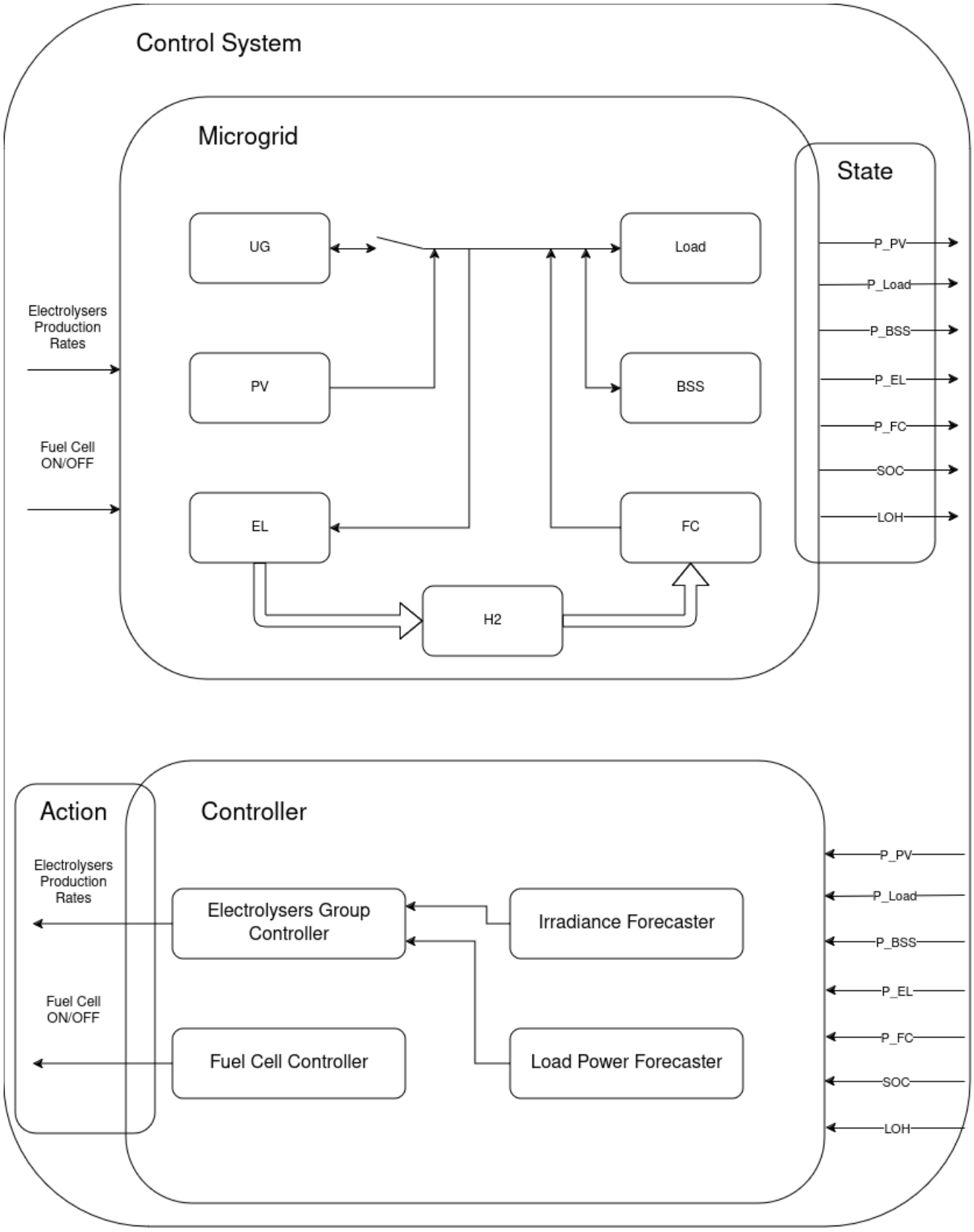
```
FuelCellController.step(plantState)
ElectrolysersGroupController.step(plantState)
action ← structure(FuelCellController.action, ElectrolysersGroupController.action)
```

Algorithm 4 FuelCellControllerStep(microgridState)

```
fuelCellOn ← False
time ← microgridState.time
if time is workTime then
    fuelCellOn ← True
end if
action ← structure(fuelCellOn)
```

Algorithm 5 ElectrolysersGroupControllerStep(microgridState, IrradianceForecaster, LoadPowerForecaster)

```
electrolysersSetProductionRates ← ArrayZeros
time ← microgridState.time
if time is workTime then
    if microgridState.soc > 0.2 and microgridState.loh < 0.99 then
        electrolysersSetProductionRates ← microgridState.electrolysersSetProductionRates
        if longTerm then                                                    ▷ longTerm
            IrradianceForecaster.step(time, microgridState.irradiance)
            LoadPowerForecaster.step(time, microgridState.loadPower)
            irradianceForecast ← IrradianceForecaster.forecast
            loadPowerForecast ← LoadPowerForecaster.forecast
            if validateForecasts then
                solarPowerForecast ← IRRADIANCE-TO-PV · irradianceForecast
                remainingPowerForecast ← solarPowerForecast-loadPowerForecast
                requiredNumberOfSwitchedOnElectrolysers ← round(remainingPowerForecast[0]/EL-MAX-
POWER)
                requiredNumberOfSwitchedOnElectrolysers ←
min(max(requiredNumberOfSwitchedOnElectrolysers, 0), ELECTROLYSERS-NUMBER)
                electrolysersSetProductionRates ←
switchProductionRatesOfElectrolysers(requiredNumberOfSwitchedOnElectrolysers,
electrolysersSetProductionRates, microgridState.electrolysersSwitchNums)
            end if
        end if
                                                                            ▷ shortTerm
        remainingPower ← IRRADIANCE-TO-PV · microgridState.irradiance - microgridState.loadPower
        totalSetPoint ← remainingPower / EL-MAX-POWER
        totalSetPoint ← min(max(totalSetPoint, 0), ELECTROLYSERS-NUMBER)
        electrolysersSetProductionRates ←
distributeProductionRatesOfSwitchedOnElectrolysers(electrolysersSetProductionRates, totalSetPoint)
    end if
end if
action ← structure(electrolysersSetProductionRates)
```


Control System
Microgrid
State
UG
Load
Electrolysers Production Rates
PV
BSS
Fuel Cell ON/OFF
EL
FC
H2
P_PV
P_Load
P_BSS
P_EL
P_FC
SOC
LOH
Action
Controller
Electrolysers Production Rates
Electrolysers Group Controller
Irradiance Forecaster
Fuel Cell ON/OFF
Fuel Cell Controller
Load Power Forecaster
P_PV
P_Load
P_BSS
P_EL
P_FC
SOC
LOH